

11 Level 3 Tools: OpenHands and Aider

With self-driving cars, there are five standardized levels that express the extent of support: assisted driving, partial automation, conditional automation, high automation, and full automation driving. Some AI publications have begun to apply this concept to autonomous coding as well (see [Table 11.1](#)).

Level	Examples	Description
0	–	Human coding without AI support
1	GitHub Copilot	Code completion: Completion of code snippets
2	ChatGTP	Code creation: Targeted programming of entire functions
3	OpenHands, Aider	Supervised automation: Human high-level instructions that the AI tool executes independently in several steps; human control and troubleshooting
4	–	Full automation: Same as supervised automation, but without the need for human control
5	–	Full autonomy: The AI tool sets the goals itself

Table 11.1 Different Levels of AI Support for Coding

There is currently no standard for the autonomy of AI coding. The level names and descriptions vary greatly depending on the source. We’ve adopted the terminology from “Levels of AGI for Operationalizing Progress on the Path to AGI” (2023, <https://arxiv.org/abs/2311.02462>). Even if it’s unclear whether this classification will prevail in the long term, the following seems appropriate to us for the time being: <https://sourcegraph.com/blog/levels-of-code-ai>.

Even the assignment of tools such as GitHub Copilot or ChatGPT to levels 1 and 2 is problematic. Of course, you can also use chat-based tools for level 1 tasks and only ask for code details. But chat-based tools are increasingly able to answer relatively complex prompts (“Create a minimal framework for a REST API with Python and Flask”). Conversely, GitHub Copilot can do much more than just code completion. In this respect, both tools (and many other AI tools with similar functions) are somewhere between level 1 and level 2. However, the actual implementation, that is, the creation of the code files, remains a human task in any case.

In this chapter, we want to present two software projects that correspond to level 3. You can ask *OpenHands* or *Aider* to fix a specific problem in a code file or to set up all the files required to implement a new function, preferably including a Git commit. Your instructions are much more abstract than at level 1 and 2 and are comparable to work assignments for a junior developer. You don’t concern yourself with the details of implementation. So, you don’t scrutinize every variable name, every loop, and so on. What you’ll actually do, however, is test the resulting code. You still need specialist knowledge to assess the basic usefulness of the code, recognize errors, and formulate clear instructions on how the code is to be developed further.

At this point, note that although *OpenHands* and *Aider* are great tools with a lot of potential, they are far from perfect. So don’t expect miracles! Nevertheless, we consider this chapter to be important because it shows the direction in which AI tools for coding are currently developing. GitHub Copilot or ChatGPT are by no means the end of the line!

AI tools that correspond to levels 4 or 5 are difficult to imagine from today’s perspective. Level 4 would be as if the company boss or head of department were to instruct the development team (today) or an AI tool (in the future) to develop a new program that performs certain tasks. The company management itself has no idea about programming and isn’t even interested in which language or which framework is used. The only decisive factor is that customers are satisfied with the product. From your point of view as a reader, it’s very doubtful whether a functioning level 4 tool is desirable at all: it would cause large parts of the IT labor market to collapse.

At level 5, the AI would also take over the formulation of the task. AI might be responsible for the success of a software company. Based on customer feedback, it recognizes a problem and commissions a solution or the development of a new

component that makes it easier for customers to use. Level 5 is currently pure science fiction.

11.1 OpenHands

OpenHands (formerly OpenDevin) is developed as an open-source project with the free MIT license on GitHub. The current version 0.16 was released in December 2024; the community is very active, as evidenced by 38,800 GitHub stars. Our tests, however, refer to version 0.9.

In contrast to the working techniques with AI assistants presented so far, OpenHands can access the file system and manage files and folders there (provided you grant this type of access). What at first sounds like a minor piece of information has great potential: the AI tool can manage entire projects, create files, and compile programs independently. Your AI assistant would no longer be limited to chat or hints in the integrated development environment (IDE), but could work independently.

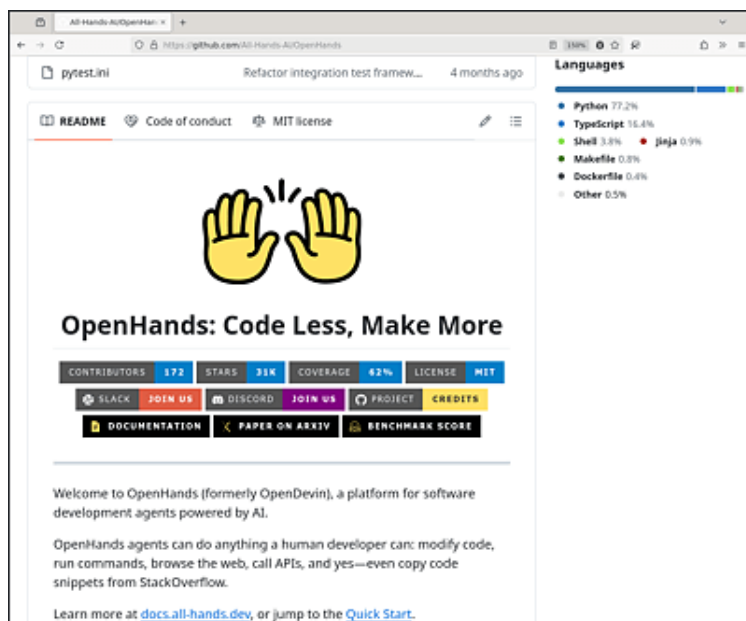


Figure 11.1 The GitHub Page of OpenHands

For example, in combination with Docker containers, it's possible to test and improve the generated source code in a secure environment. You can already see where the journey is going: For example, if an error is found when compiling a file, the AI software can use the error message to attempt to resolve the problem automatically. For each step, OpenHands creates a prompt that is sent to an LLM. The response gets analyzed and converted into commands if necessary.

In this way, commands could be given to the AI assistants on a more abstract level. Instead of working through the individual steps yourself, you could also request that a React app is created that displays PDFs, for example. Your AI assistant should then independently install the necessary packages, create the folder structure and files, start the web server, create test users and test the API using `curl`, and so on.

We've tried this out in [Section 11.2](#) and tested whether OpenHands can independently create a small web application with a few users.

The Right Large Language Model (LLM) for OpenHands

When trying out OpenHands, you potentially generate a large number of requests to a language model. If you use a cloud provider for this, it can quickly become expensive (we've been there too). OpenHands also supports local LLMs, but our attempts with `llama3.1`, `codegemma`, or `deepseek-coder` were very disappointing. None of them returned any useful results. Using proprietary models such as `gpt-4o` (the default setting in OpenHands), we were able to achieve small successes, which we want to present to you here.

11.1.1 Installation

To exploit the full potential of OpenHands, the program must also be able to install software. For example, if OpenHands is to create and test software in the Go language, it requires the compiler and the Go modules this program uses. If these components were to be installed on your computer's operating system at every attempt, your computer would soon be pretty messed up.

Docker containers provide an ideal solution in this context: OpenHands can do what it wants in a *sandbox container*. When you exit OpenHands, the container (with all the installed software) gets deleted. The working directory in which the desired code is created is retained, of course. We assume that you have basic experience with Docker and have installed it on your computer. You should also use a terminal window in which a standard Unix shell such as `bash` or `zsh` is running. On Linux and macOS, this shouldn't be a problem, whereas on Windows, you have to use Windows Subsystem for Linux (WSL) with a Linux system.

The current OpenHands version can be started as follows:

```
WORKSPACE_BASE=$(pwd)/workspace
docker run -it \
```

```

--pull=always \
-e SANDBOX_RUNTIME_CONTAINER_IMAGE=\
    ghcr.io/all-hands-ai/runtime:0.9-nikolaik \
-e SANDBOX_USER_ID=$(id -u) \
-e WORKSPACE_MOUNT_PATH=$WORKSPACE_BASE \
-v $WORKSPACE_BASE:/opt/workspace_base \
-v /var/run/docker.sock:/var/run/docker.sock \
-p 3000:3000 \
--add-host host.docker.internal:host-gateway \
--name openhands-app-$(date +%Y%m%d%H%M%S) \
ghcr.io/all-hands-ai/openhands:0.9

```

Note that a `workspace` folder will be created in the current directory in which your new software will be developed. If this folder already exists, it will be integrated together with the existing content. You can change this directory by adjusting the `WORKSPACE_BASE` variable in the first line.

Including the `/var/run/docker.sock` socket allows the container to control the Docker daemon. OpenHands requires this setting so that the sandbox container can be started by the application. However, controlling the Docker daemon also gives the container access to all other Docker resources on your computer. So don't start OpenHands on a system on which important Docker applications are running productively.

At startup, a container is derived from the current Docker image, which is assigned the name `openhands-app-XXXXX`, where `XXXXX` is replaced with the current date and time. This kind of naming ensures that you can easily find the container of an aborted attempt at a later stage. The container contains all log files that were created during the test, which can be useful for analysis.

The web application then runs on `http://localhost:3000`. The sandbox container is started automatically as soon as you load the web interface and is assigned the name `openhands-sandbox-YYYYY`, where `YYYYY` stands for a randomly generated, unique ID. After the successful start, you can display the two containers with the Docker subcommand `ps` (the output has been specially formatted due to the long names):

```
> docker ps --format '{{.Image}} {{.Names}}'
```

```

ghcr.io/all-hands-ai/runtime:0.9-nikolaik  openhands-sandbox...
ghcr.io/all-hands-ai/openhands:0.9  openhands-app-20240910163059

```

11.1.2 The Web Interface

Once you've successfully completed the installation, you can start using OpenHands. To do this, open `http://localhost:3000` in your web browser with the OpenHands web interface.

The chat with OpenHands is created on the left-hand side. There, you enter your requirements, and OpenHands explains the steps that are carried out. You can view the generated code (and any other files and folders) in the top-right area of the browser window. Below this is an interactive terminal, which OpenHands itself also makes use of. This is a shell in a sandbox container.

For OpenHands to process your instructions, you must first configure the LLM and the *agent*, whereby the agent takes over the communication between the LLM and the rest of the software. The corresponding configuration dialog ap-

pears the first time the web interface is opened and can be called again at any time using the screw symbol at the bottom right.

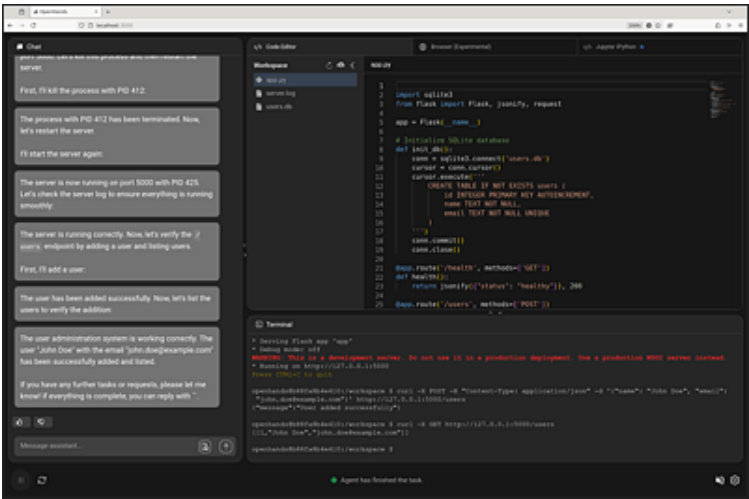


Figure 11.2 The OpenHands Web Interface Currently Only Available in “Dark Mode”

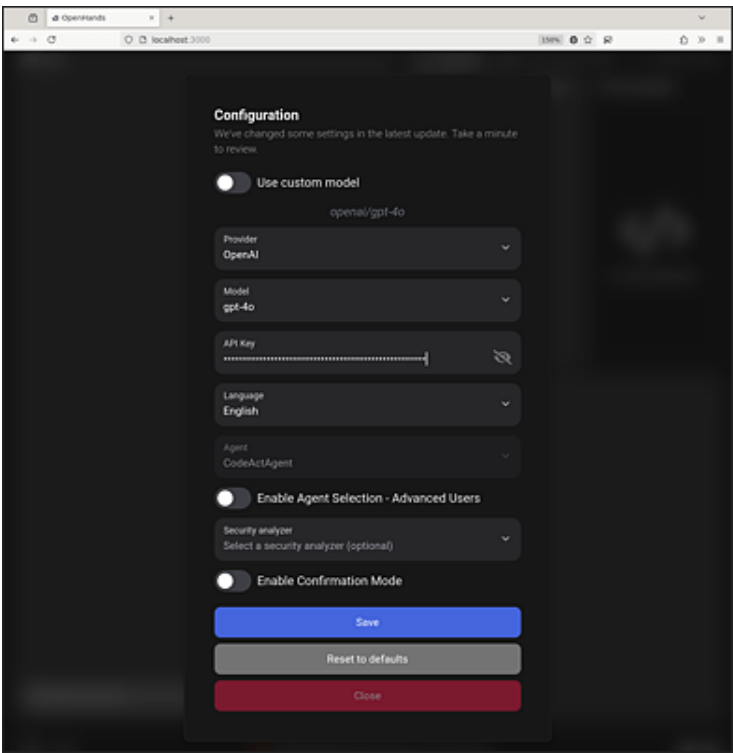


Figure 11.3 Default Configuration Settings of OpenHands

To use the default gpt-4o model from OpenAI, you must enter the API key that you’ve previously created in the settings of your OpenAI account (see [Chapter 10](#)). The default CodeActAgent is used as the agent for the gpt-4o model, which brings us to the topic of local LLMs. As already mentioned, our successes with local language models were extremely limited. The OpenHands online help also confirms this observation and refers to the *GPT-4* and *Claude 3* models as currently the best partners for OpenHands.

Perhaps this situation will have improved by the time you hold the book in your hands and you also want to give the more cost-effective variant with local models a chance. OpenHands supports the Ollama API, which we've already presented in [Chapter 9, Section 9.6](#). For access to your local Ollama models to work, you must set the `LLM_BASE_URL` variable when starting the container:

```
docker run \
    ...
    -e LLM_BASE_URL="http://host.docker.internal:11434" \
    ...
    ghcr.io/all-hands-ai/openhands:0.9
```

If `ollama` isn't running on your local computer, but on a computer in the local area network (LAN; as was the case in our tests), you need to enter the Domain Name System (DNS) name of the computer on which the service is running instead of `host.docker.internal`. Make sure that the `openhands` app container has access to port 11434 on this computer and that no firewall is interfering. You can then enter the `ollama/llama3.1` string in the configuration dialog under *Model* (if you want to use the `llama3.1` LLM from your local installation). Note that switching to a model of a cloud provider, such as `gpt-4o` from OpenAI, only works if you restart the container and don't define the `LLM_BASE_URL` variable.

During our tests, we found the error messages in the browser chat not very helpful. OpenHands outputs a few technical error messages here, which often leads to very abbreviated, generic messages. The log output in the terminal window in which we started the Docker container was much more helpful. These messages are very detailed and usually provide quick information about the actual problem. Here is an excerpt from the log file of the Python web application:

```
CodeActAgent LEVEL 0 LOCAL STEP 13 GLOBAL STEP 13

06:37:31 - openhands:INFO: llm.py:486 - Cost: 0.04 USD |
    Accumulated Cost: 0.43 USD
Input tokens: 7757
Output tokens: 53

06:37:31 - ACTION
**CmdRunAction (source=EventSource.AGENT)**
THOUGHT: Let's use `netstat` to find the process using port 5000
and then kill it.
```

First, let's find the process ID (PID) using port 5000.

```
COMMAND:
netstat -tuln | grep :5000
06:37:31 - openhands:INFO: runtime.py:359 - Awaiting session
06:37:31 - openhands:INFO: runtime.py:263 -
-----Container logs:-----
| INFO:      172.17.0.1:53856 - "GET /alive HTTP/1.1" 200 OK
| INFO:      172.17.0.1:53856 - "POST /execute_action HTTP/1..."
-----
06:37:31 - openhands:INFO: session.py:139 - Server event
06:37:31 - OBSERVATION
**CmdOutputObservation (source=EventSource.AGENT, exit code=1)**
bash: netstat: command not found

[Python Interpreter: /openhands/poetry/openhands-ai-504_aChf-p...
openhands@200f871dd6cf:/workspace $
06:37:31 - openhands:INFO: session.py:139 - Server event
```

For this reason, you should always keep an eye on the terminal window with these log messages when experimenting with OpenHands.